

C++ Standard Template Library (STL)

1. Introduction

The **Standard Template Library (STL)** is a collection of **inbuilt generic class templates and functions** in C++ that provide **ready-to-use data structures and algorithms**.
It enhances productivity, reusability, and safety by avoiding manual implementation of common data structures.

Key Components of STL

1. **Containers** – Data structures like vector, list, set, map, stack, etc.
2. **Iterators** – Objects for sequentially accessing elements in containers.
3. **Algorithms** – Generic functions for operations like sort, count, reverse, max, min, etc.

2. Difference Between STL Containers and Normal Arrays

Feature	Normal Array	STL Containers
Nature	Static (fixed size)	Dynamic (size can grow or shrink)
Type Safety	Not type-safe	Strongly type-safe
Memory Management	Manual	Automatic
Functions/Methods	Limited (index access only)	Rich set of member functions
Resizing	Not possible	Possible (push_back(), resize())
Performance	Fast but unsafe	Slightly slower but safe
Flexibility	Stores only one type of data	Template-based — can store any data type

3. Containers Overview

3.1 Vector

A **dynamic array** that grows automatically when new elements are added.

Syntax:

```
#include <vector>
vector<int> v;
```

Key Features:

- Random access supported (v[i])
- Contiguous memory allocation
- Automatic resizing
- Fast insertion/deletion at the end

Important Functions:

Function	Description
<code>push_back()</code>	Adds an element at the end
<code>pop_back()</code>	Removes the last element
<code>size()</code>	Returns the current number of elements
<code>capacity()</code>	Returns the total allocated memory slots
<code>reserve(n)</code>	Preallocates memory for n elements
<code>resize(n)</code>	Changes the number of elements
<code>clear()</code>	Removes all elements
<code>begin(), end()</code>	Returns iterator to start and end
<code>rbegin(), rend()</code>	Reverse iterators

Example:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<char> v(10);
    for (int i = 0; i < 10; i++) v[i] = i + 'a';
    for (int i = 0; i < v.size(); i++) cout << v[i] << " ";

    // Add more elements
    for (int i = 0; i < 10; i++) v.push_back(i + 10 + 'a');

    cout << "\nNew size: " << v.size() << endl;
    return 0;
}
```

3.2 Vector with User-Defined Class Objects

When objects are stored in a vector, **copies of those objects** are stored — not references.

```
#include <iostream>
#include <vector>
using namespace std;

class MyClass {
    int num;
public:
    MyClass(int k) : num(k) { cout << "Parameterized constructor\n"; }
    MyClass(const MyClass& ref) { cout << "Copy constructor\n"; num = ref.num; }
    void disp() { cout << num << endl; }
    ~MyClass() { cout << "Destructor called\n"; }
};

int main() {
    vector<MyClass> v;
    MyClass m1(10), m2(20);
    v.push_back(m1);
    v.push_back(m2);
    for (auto& obj : v) obj.disp();
}
```

Note: Copy constructor is called during `push_back()` since vectors store **copies** of objects.

3.3 Vector with Inheritance (Polymorphism)

```
#include <iostream>
#include <vector>
using namespace std;

class Base { public: virtual void disp() { cout << "Base\n"; } };
class Sub : public Base { public: void disp() { cout << "Sub\n"; } };
class Sub1 : public Base { public: void disp() { cout << "Sub1\n"; } };

int main() {
    vector<Base*> v;
    Base b; Sub s; Sub1 s1;
    v.push_back(&b); v.push_back(&s); v.push_back(&s1);

    for (auto p : v) p->disp();
}
```

Explanation:

- Polymorphism works when storing pointers (Base*).
- Virtual functions ensure correct overridden function calls.

3.4 Vector of Objects (Slicing Issue)

If objects are stored directly (not as pointers), only the **Base** part of derived objects is stored (object slicing).

```
vector<Base> v;
v.push_back(b);
v.push_back(s);
v.push_back(s1);
```

Here, derived members are sliced off.

3.5 Vector File I/O Example

Vectors can be directly written to or read from binary files using `.data()` and `.size()`.

```
f.write(v.data(), v.size());
f.read(temp.data(), temp.size());
```

- `v.data()` → Pointer to internal array
- `v.size()` → Number of bytes to write or read

3.6 Vector Specialization

`vector<bool>` is optimized to store boolean values compactly (1 bit per value).

It behaves like a **bit vector**, not a normal vector.

```
vector<bool> v;
for (int i = 0; i < 10; i++)
    v.push_back(i % 2);

size_t cnt = count(v.begin(), v.end(), true);
cout << "True values: " << cnt << endl;
```

3.7 Algorithms with Vectors

STL provides predefined algorithms for operations such as count, reverse, sort, etc.

Example:

```
vector<int> v = { 1, 2, 3, 4, 5 };
reverse(v.begin(), v.end());
count(v.begin(), v.end(), 3);
```

4. List Container

A **list** is a **doubly linked list** implementation provided by STL.

It allows fast insertion and deletion anywhere in the list but does not support random access.

Header:

```
#include <list>
```

Features:

- Dynamic size
- Fast insertion/deletion
- Sequential access only (no random indexing)

Important Functions:

Function	Description
push_front()	Insert at beginning
push_back()	Insert at end
pop_front()	Remove from beginning
pop_back()	Remove from end
insert()	Insert at given position
erase()	Delete at given position
sort()	Sorts elements
reverse()	Reverses list order
unique()	Removes duplicates
merge()	Merges two sorted lists
splice()	Moves elements from one list to another

Internal Implementation

- Each element is stored in a **node** containing:
 - Value (`_Myval`)
 - Pointers to next and previous nodes (`_Next, _Prev`)
- The list maintains a **head node** that connects to the first and last elements, forming a circular doubly linked structure.

Iterators in List

Iterators are bidirectional, allowing traversal in both directions.

Example:

```
list<int> l;  
for (int i = 1; i <= 5; ++i) l.push_back(i);  
  
for (auto it = l.begin(); it != l.end(); ++it)  
    cout << *it << " ";
```

5. Set Container

A **set** stores unique elements in **sorted order** automatically.

```
#include <set>
using namespace std;

int main() {
    set<int> s;
    for (int i = 10; i > 0; i--) s.insert(i);
    s.insert(10); // duplicate ignored

    for (auto it = s.begin(); it != s.end(); ++it)
        cout << *it << " ";
}
```

Characteristics:

- Elements are unique
 - Automatically sorted
 - Based on balanced binary search tree (Red-Black Tree)
 - Fast lookup and insertion ($O(\log n)$)
-

6. Map Container

A **map** stores data in **key-value pairs** in sorted order by key.
Each key is unique.

Syntax:

`map<KeyType, ValueType> m;`

Example:

```
#include <map>
#include <iostream>
using namespace std;

int main() {
    map<char, int> m;
    for (int i = 0; i < 26; i++)
        m.insert(pair<char, int>('A' + i, 65 + i));

    char ch;
    cout << "Enter key: ";
    cin >> ch;

    auto it = m.find(ch);
    if (it != m.end())
        cout << "Value: " << it->second;
    else
        cout << "Key not found";
}
```

Notes:

- Keys are automatically sorted.
 - Access value using `it->second`.
 - Searching is $O(\log n)$ due to tree-based implementation.
-

7. Stack Container Adapter

A **stack** is a container adapter that follows the **LIFO (Last-In-First-Out)** principle.

Syntax:

```
#include <stack>
stack<Type> s;
```

Important Functions:

Function	Description
push()	Add element to top
pop()	Remove top element
top()	Access top element
empty()	Check if stack is empty
size()	Number of elements

Example with Structure:

```
#include <iostream>
#include <stack>
using namespace std;

struct emp { char name[20]; int age; };
ostream& operator<<(ostream& o, emp& m) {
    return o << m.name << "\t" << m.age << endl;
}

int main() {
    emp e1 = { "abc", 30 }, e2 = { "xyz", 10 };
    stack<emp> s;
    s.push(e1); s.push(e2);
    cout << "Size: " << s.size() << endl;
    while (!s.empty()) {
        cout << s.top();
        s.pop();
    }
}
```

8. String Class

The string class in C++ is a safer alternative to C-style null-terminated character arrays.

Advantages:

- No buffer overflow issues
- Dynamic resizing
- Supports operators like +, =, ==
- Built-in functions for manipulation

Example:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string s1("abc"), s2("xyz");
    string s3 = s1 + s2;
    cout << s3 << endl;
}
```

```

    for (char c : s3) cout << c << endl;

    string* ptr = new string("I am on heap");
    cout << *ptr << endl;
    delete ptr;
}

```

9. Algorithms and Iterators

Iterators act like pointers to container elements, providing access and traversal.

Common Iterator Operations:

- `begin()` → First element
- `end()` → Past-the-last element
- `++it` → Move forward
- `--it` → Move backward

Algorithms:

- Work with any iterator-based container.
- Example functions: `count()`, `sort()`, `reverse()`, `find()`, `max_element()`, etc.

10. Summary

Concept	Description
STL	Collection of generic, reusable templates for data structures and algorithms
Containers	Objects that store data (vector, list, set, map, etc.)
Iterators	Provide sequential access to container elements
Algorithms	Functions operating on container data
Vector	Dynamic array (contiguous memory)
List	Doubly linked list
Set	Unique sorted elements
Map	Key-value pairs, sorted by key
Stack	LIFO container adapter
String	Dynamic text handler class
Dynamic vs Static	STL containers are dynamic; arrays are static
Efficiency	STL automates memory management, ensuring safety and flexibility

Key Advantages of STL

1. **Reusability** – Avoids reinventing data structures.
2. **Type Safety** – Template-based type checking.

3. **Flexibility** – Works with any data type.
 4. **Efficiency** – Optimized for performance.
 5. **Extensibility** – Can combine containers, iterators, and algorithms seamlessly.
-

